COMP102: Data Structures with Python

Linked Lists

Madani Badaoui
March 30, 2026

Today's Roadmap

Part I — Foundations

1. Why Linked Lists?
2. Nodes & References
3. The `Node` Class

Part II — Singly Linked Lists

4. Building the List
5. Traversal & Iteration
6. Insertion (head / tail / middle)
7. Deletion (head / tail / middle)

Part III — Doubly Linked Lists

8. Two-way References
9. Insertion & Deletion

Part IV — Analysis & Practice

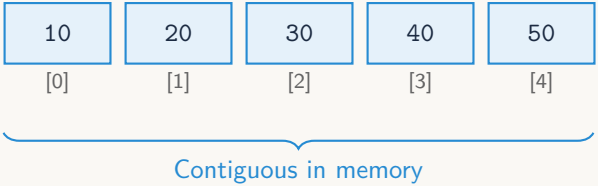
10. Complexity Comparison
11. Common Patterns & Pitfalls

Part I: Foundations

Why Do We Need a New Structure?

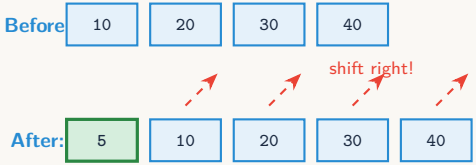
Recall: Python Lists (Arrays)

Python's built-in `list` stores elements in **contiguous memory**.



The Problem with Contiguous Storage

Inserting at the beginning:



Cost

Inserting at index 0 requires **shifting every element** one position to the right.

Time: $O(n)$

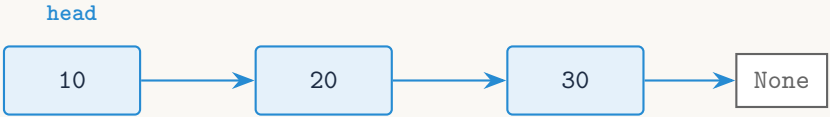
Question

Can we insert at the front in $O(1)$?

Enter: The Linked List

Key Idea

Instead of storing elements side-by-side, store each element in its own **node** and connect nodes with **references** (pointers).



Elements are **scattered** in memory — connected by references

Linked List vs. Python List — Quick Comparison

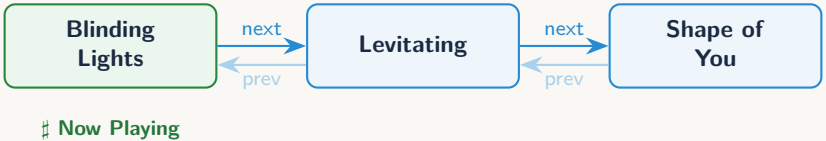
Feature	Python List (Array)	Linked List
Memory Layout	Contiguous	Scattered
Access by Index	$O(1)$	$O(n)$
Insert at Head	$O(n)$	$O(1)$
Insert at Tail	$O(1)$ amortized	$O(n)$ or $O(1)^*$
Delete at Head	$O(n)$	$O(1)$
Memory Overhead	Low	Higher (pointers)

* $O(1)$ if we maintain a `tail` reference.

Real-Life Example: Your Music Playlist

Something You Use Every Day

When you listen to music on **Spotify**, **YouTube Music**, or **Apple Music**, the queue of songs is managed like a **linked list**.



Playlist as a Linked List — The Mapping

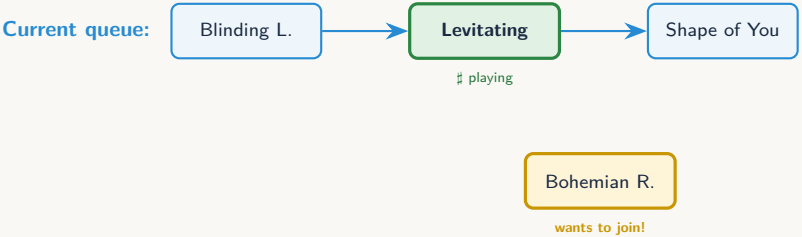
Playlist Action	Linked List Operation	Cost
Press **Next** ▶▶	Follow `current.next`	$O(1)$
Press **Previous** ◀◀	Follow `current.prev`	$O(1)$
Add song to "Up Next"	Insert after current node	$O(1)$
Remove a song from queue	Delete node, relink neighbors	$O(1)$
Shuffle the playlist	Rearrange the node links	$O(n)$
Jump to song #42	Traverse from head to index	$O(n)$

Key Insight

The most **frequent** actions (next, previous, add to queue) are all $O(1)$. That is *exactly* why a linked list fits here!

Playlist: Adding a Song to "Up Next"

Scenario: You are listening to "Levitating" and add "Bohemian Rhapsody" to play next.



Playlist: Adding a Song to "Up Next"

Scenario: You are listening to "Levitating" and add "Bohemian Rhapsody" to play next.

After insert:

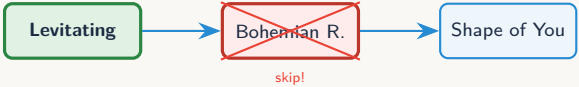
What happened?

`new_song.next = current.next` then `current.next = new_song`

No shifting. No copying. Just **two pointer updates** — $O(1)$.

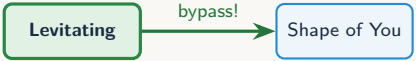
Playlist: Skipping a Song

Scenario: You skip "Bohemian Rhapsody" (remove it from the queue).



Playlist: Skipping a Song

Scenario: You skip "Bohemian Rhapsody" (remove it from the queue).



This is **exactly** the delete-node operation: `prev.next = prev.next.next` — skip over the removed song.

Playlist: A Mini Implementation

```python
class Song:
    """A node in our playlist (doubly linked)."""
    def __init__(self, title, artist):
        self.title = title
        self.artist = artist
        self.prev = None
        self.next = None
class Playlist:
    def __init__(self):
        self.current = None  # the song now playing
    def play_next(self):
        """Skip to next song. O(1)"""
        if self.current and self.current.next:
            self.current = self.current.next
        print(f"Now playing: {self.current.title}")
    def play_prev(self):
        """Go back to previous song. O(1)"""
        if self.current and self.current.prev:
            self.current = self.current.prev
        print(f"Now playing: {self.current.title}")

```

Playlist: Adding to “Up Next”

```python
def add_next(self, title, artist):
    """Insert a song right after the current one. O(1)"""
    new_song = Song(title, artist)
    new_song.prev = self.current
    new_song.next = self.current.next
    if self.current.next:
        self.current.next.prev = new_song
    self.current.next = new_song
    print(f"Added '{title}' to Up Next")

```

```python
# Example usage
p = Playlist()
# ... add initial songs ...
p.add_next("Bohemian Rhapsody", "Queen")
p.play_next()   # Now playing: Bohemian Rhapsody
p.play_prev()   # Now playing: Levitating

```

Takeaway

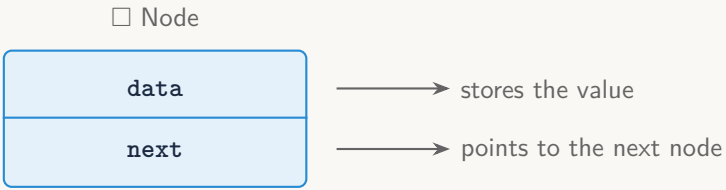
Every method is $O(1)$ — no loops, no shifting, just pointer updates. **This is the power of linked lists.**

What Is a Node?

Definition

A **node** is a container that holds:

1. A piece of **data** (the element itself)
2. A **reference** to the next node (or `None`)



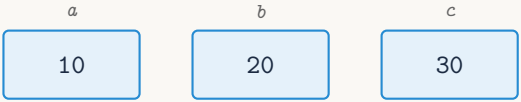
The `Node` Class in Python

```python
class Node:
    """A single node in a singly linked list."""
    def __init__(self, data):
        self.data = data   # the stored value
        self.next = None   # reference to the next node

```

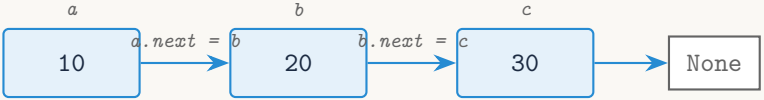
```python
# Creating individual nodes
a = Node(10)
b = Node(20)
c = Node(30)
# Linking them manually
a.next = b
b.next = c
# c.next remains None (end of list)

```

Visualizing Manual Linking

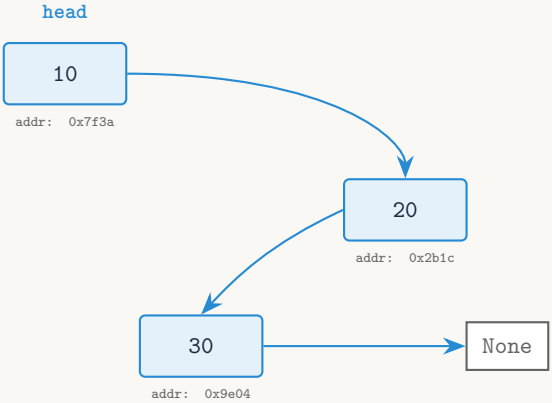
Step 1: Create nodes



Step 2: Link them



Nodes in Memory — The Real Picture



Nodes live at **arbitrary** addresses.
The `next` reference is what
keeps the chain intact.

Part II: Singly Linked Lists

The `SinglyLinkedList` Class — Skeleton

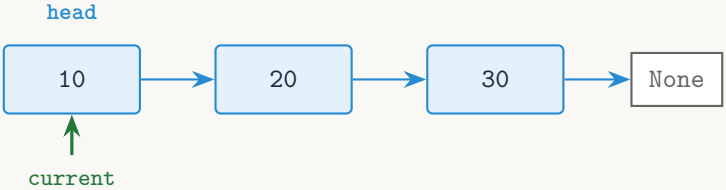
```python
class SinglyLinkedList:
    """A singly linked list."""
    def __init__(self):
        self.head = None    # reference to the first node
        self.size = 0       # number of elements

```

(empty list)

Traversal — Walking the Chain

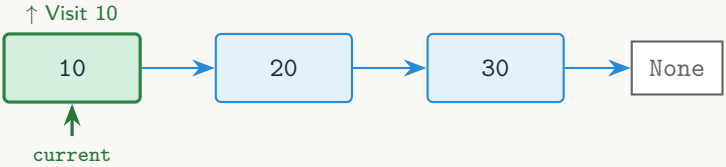
Algorithm

Start at `head`. Visit each node by following `.next` until you reach `None`.

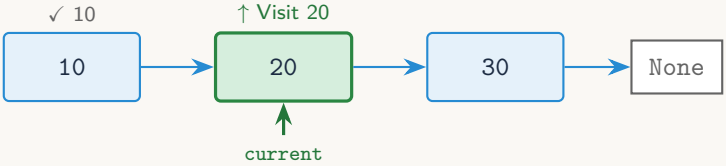


`current` starts at `head`, then hops to each `.next` in turn.

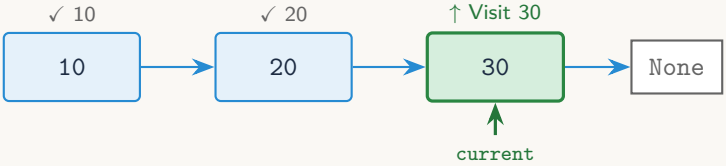
Traversal — Step by Step



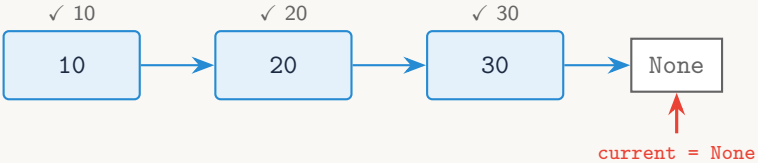
Traversal — Step by Step



Traversal — Step by Step



Traversal — Step by Step



STOP — traversal complete!

Traversal — Python Code

```python
def display(self):
    """Print all elements in the list."""
    current = self.head
    elements = []
    while current is not None:
        elements.append(str(current.data))
        current = current.next
    print(" -> ".join(elements) + " -> None")

```

Output

```
10 -> 20 -> 30 -> None
```

Making the List Iterable — `__iter__`

```python
def __iter__(self):
    """Allow: for item in linked_list"""
    current = self.head
    while current is not None:
        yield current.data
        current = current.next

```

```python
# Usage
ll = SinglyLinkedList()
# ... add some elements ...
for value in ll:
    print(value)

```

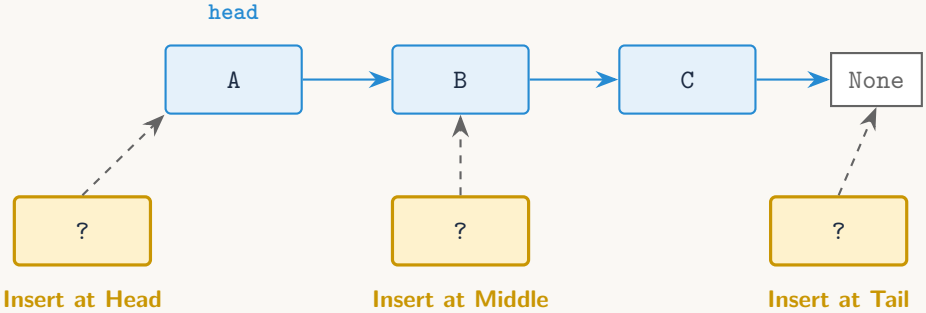
Why `yield`?

The `yield` keyword turns `__iter__` into a **generator**, producing one value at a time without building a full list in memory.

Other Useful Dunder Methods

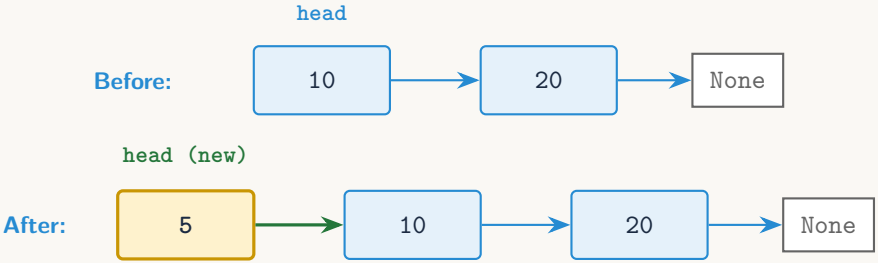
```python
def __len__(self):
    """Allow: len(linked_list)"""
    return self.size
def __contains__(self, target):
    """Allow: target in linked_list"""
    for data in self:          # uses __iter__
        if data == target:
            return True
    return False
def __repr__(self):
    """Readable string representation."""
    items = " -> ".join(str(x) for x in self)
    return f"SinglyLinkedList([{items}])"

```

Insertion — Three Cases

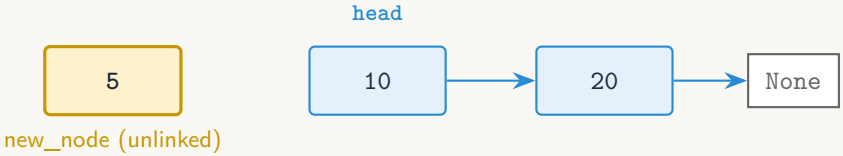


Insert at Head — Concept

Goal: Add a new node *before* the current head.

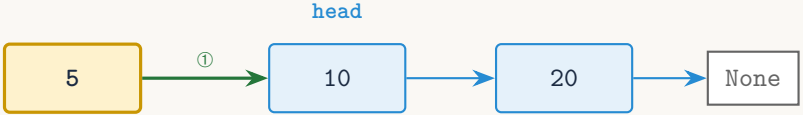


Insert at Head — Two Critical Steps



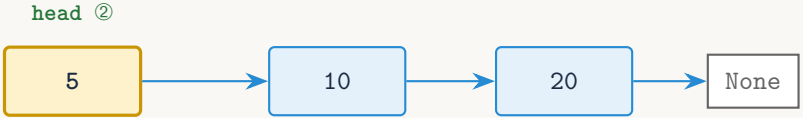
Insert at Head — Two Critical Steps

Step 1 `new_node.next = self.head`



Insert at Head — Two Critical Steps

Step 2: `self.head = new_node`



Insert at Head — Two Critical Steps

Order Matters!

If you do Step 2 first, you **lose** the reference to the old head!

Insert at Head — Python Code

```python
def insert_at_head(self, data):
    """Insert a new node at the beginning. O(1)"""
    new_node = Node(data)
    new_node.next = self.head   # Step 1: link to old head
    self.head = new_node        # Step 2: update head
    self.size += 1

```

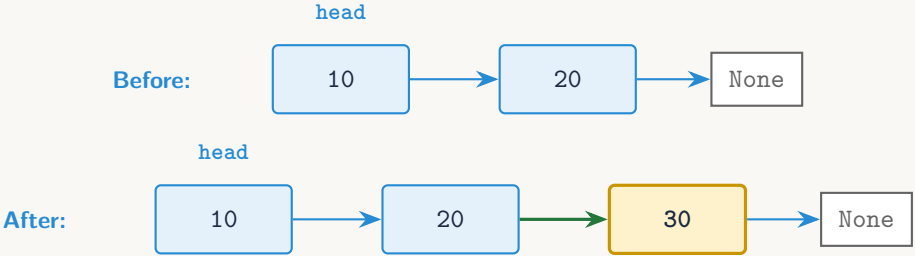
Complexity

Time: $O(1)$ — no traversal needed!

Space: $O(1)$ — only one new node created.

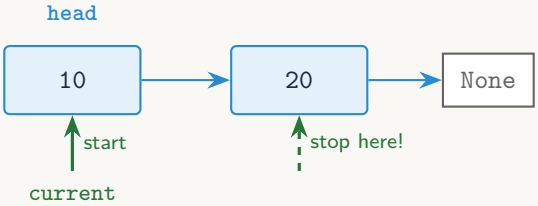
Insert at Tail — Concept

Goal: Add a new node *after* the last node.



Challenge: We must *traverse* to find the last node first.

Insert at Tail — Finding the Last Node



Stop when `current.next` is `None`

Key Check

We check `current.next`, *not* `current`.
We need to stop **at** the last node (not past it).

Insert at Tail — Python Code

```python
def insert_at_tail(self, data):
    """Insert a new node at the end. O(n)"""
    new_node = Node(data)
    if self.head is None:          # empty list
        self.head = new_node
    else:
        current = self.head
        while current.next is not None:
            current = current.next   # walk to last node
        current.next = new_node      # attach new node
    self.size += 1

```

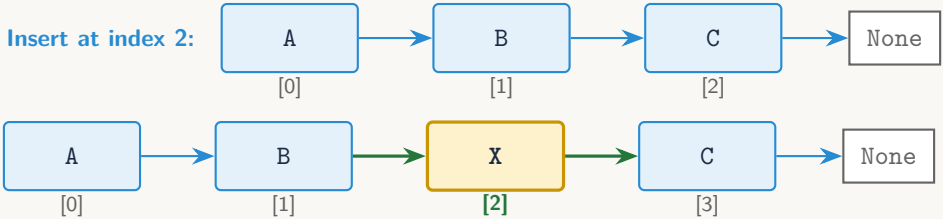
Complexity

Time: $O(n)$ — must traverse the entire list.

Space: $O(1)$

Insert at Index — Concept

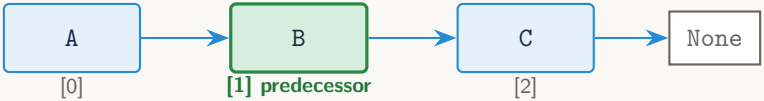
Goal: Insert a new node at position k (0-indexed).



Insert at Index — The Strategy

Key Insight

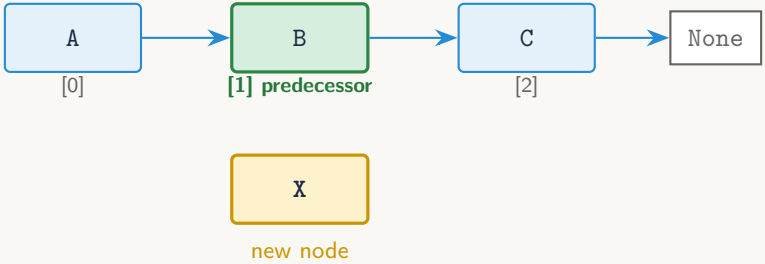
To insert at index k, we need to find the node at index $k - 1$ (the **predecessor**).



Insert at Index — The Strategy

Key Insight

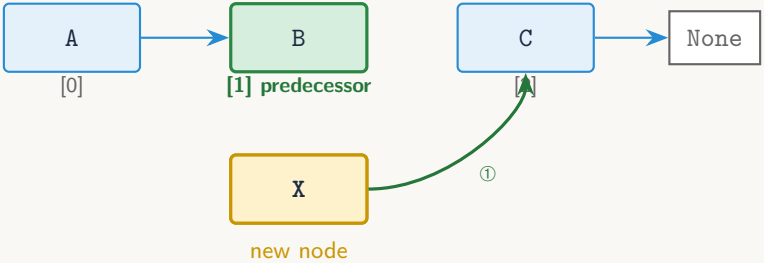
To insert at index k, we need to find the node at index $k - 1$ (the **predecessor**).



Insert at Index — The Strategy

Key Insight

To insert at index k, we need to find the node at index $k - 1$ (the **predecessor**).

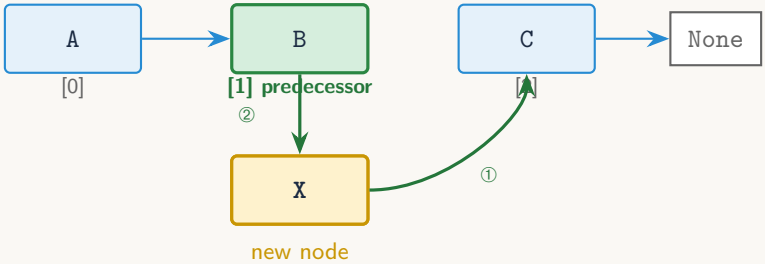


① `new_node.next = prev.next`

Insert at Index — The Strategy

Key Insight

To insert at index k, we need to find the node at index $k - 1$ (the **predecessor**).

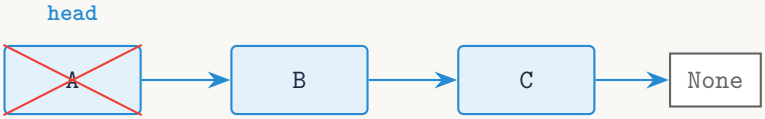


① `new_node.next = prev.next` ② `prev.next = new_node`

Insert at Index — Python Code

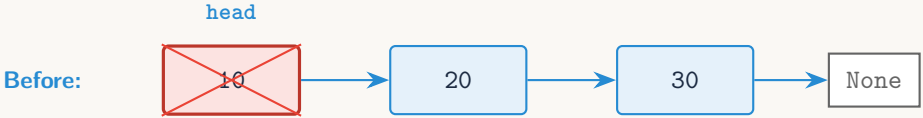
```python
def insert_at_index(self, index, data):
    """Insert at a given index. O(n)"""
    if index < 0 or index > self.size:
        raise IndexError("Index out of range")
    if index == 0:
        self.insert_at_head(data)
        return
    new_node = Node(data)
    current = self.head
    for _ in range(index - 1):        # walk to predecessor
        current = current.next
    new_node.next = current.next       # Step 1
    current.next = new_node            # Step 2
    self.size += 1
```

Deletion — Three Cases

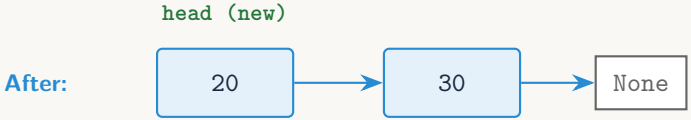


Delete Head **Delete Middle** **Delete Tail**

Delete at Head — Concept



Delete at Head — Concept



Just move `head` forward: `self.head = self.head.next`

Delete at Head — Python Code

```python
def delete_at_head(self):
    """Remove and return the first element. O(1)"""
    if self.head is None:
        raise IndexError("List is empty")
    data = self.head.data        # save the value
    self.head = self.head.next   # move head forward
    self.size -= 1
    return data
```

Complexity

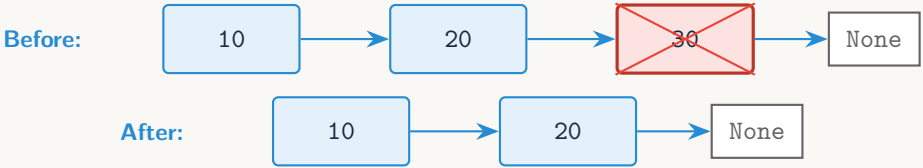
Time: $O(1)$ — just reassign one reference!

Space: $O(1)$

Garbage Collection

In Python, the old head node is **automatically freed** when no variable references it anymore.

Delete at Tail — Concept



Challenge: Must traverse to the *second-to-last* node.

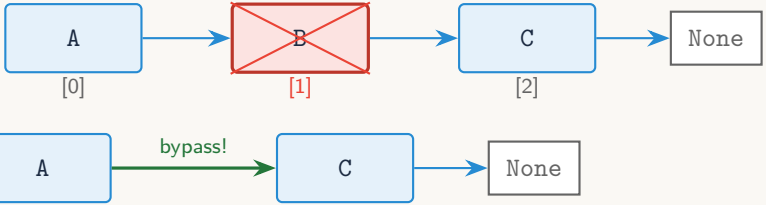
Delete at Tail — Python Code

```python
def delete_at_tail(self):
    """Remove and return the last element. O(n)"""
    if self.head is None:
        raise IndexError("List is empty")
    if self.head.next is None:     # only one node
        data = self.head.data
        self.head = None
        self.size -= 1
        return data
    current = self.head
    while current.next.next is not None:
        current = current.next     # stop at second-to-last
    data = current.next.data       # save tail value
    current.next = None            # cut the tail
    self.size -= 1
    return data
```

Delete at Index — Concept

Goal: Remove the node at index k.

Delete index 1:



`prev.next = prev.next.next` (skip over the target node)

Delete at Index — Python Code

```python
def delete_at_index(self, index):
    """Remove and return element at given index. O(n)"""
    if index < 0 or index >= self.size:
        raise IndexError("Index out of range")
    if index == 0:
        return self.delete_at_head()
    current = self.head
    for _ in range(index - 1):        # walk to predecessor
        current = current.next
    data = current.next.data          # save target value
    current.next = current.next.next  # bypass target
    self.size -= 1
    return data

```

Searching for a Value

```python
def search(self, target):
    """Return the index of target, or -1 if not found. O(n)"""
    current = self.head
    index = 0
    while current is not None:
        if current.data == target:
            return index
        current = current.next
        index += 1
    return -1

```

No Random Access

Unlike arrays, we **cannot** jump to index k directly.

Every search or access requires a walk from `head`.

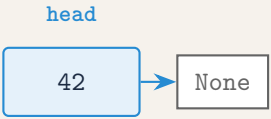
Edge Cases to Always Consider

△! Empty List

`self.head is None`

△! Single Element

`self.head.next is None`

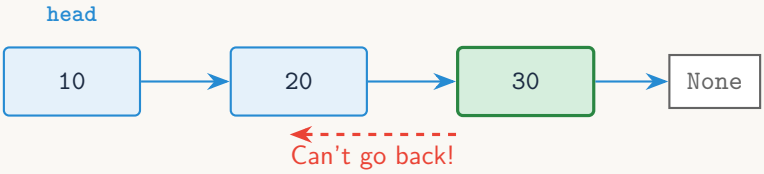


Common Bugs

1. Forgetting the empty-list check
2. `NoneType has no attribute 'next'` — accessing `.next` on `None`
3. Wrong traversal stop condition
4. Not updating `self.size`

Part III: Doubly Linked Lists

Limitation of Singly Linked Lists



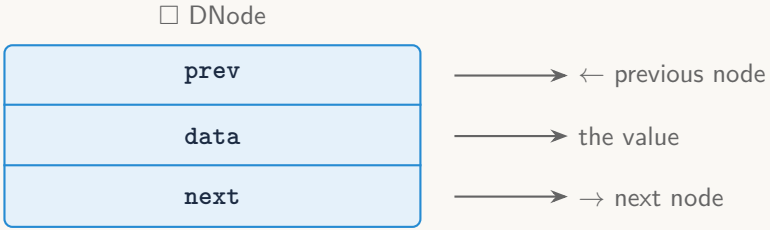
Problem

In a singly linked list, you can only move **forward**.
To delete the tail, we must traverse from the head to find the predecessor — $O(n)$.

Solution

Add a **previous** reference to each node → **Doubly Linked List**

Doubly Linked List Node



The DNode Class in Python

```python
class DNode:
    """A node in a doubly linked list."""
    def __init__(self, data):
        self.data = data
        self.prev = None    # reference to previous node
        self.next = None    # reference to next node

```

Extra Cost

Each node now uses **more memory** (one extra reference).

Trade-off: more memory $\longleftrightarrow$ faster backward traversal.

Doubly Linked List — Structure

Key Properties

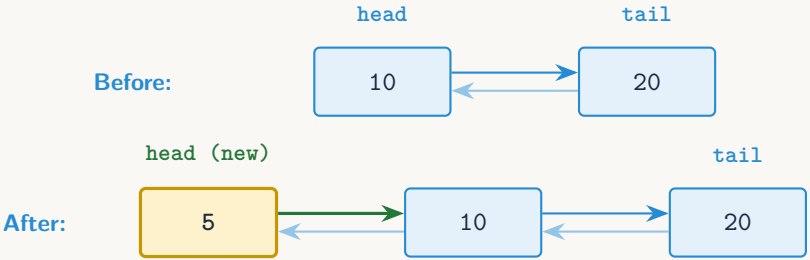
- Each node has **two** references: `prev` and `next`
- We maintain both `head` and `tail` pointers
- `head.prev = None` and `tail.next = None`

The `DoublyLinkedList` Class — Skeleton

```python
class DoublyLinkedList:
    """A doubly linked list with head and tail pointers."""
    def __init__(self):
        self.head = None
        self.tail = None
        self.size = 0

```

(empty list)

DLL — Insert at Head



Steps

① `new_node.next = self.head` ② `self.head.prev = new_node`
③ `self.head = new_node`

DLL — Insert at Head — Python Code

```python
def insert_at_head(self, data):
    """Insert at the beginning. O(1)"""
    new_node = DNode(data)
    if self.head is None:          # empty list
        self.head = new_node
        self.tail = new_node
    else:
        new_node.next = self.head  # link forward
        self.head.prev = new_node  # link backward
        self.head = new_node       # update head
    self.size += 1

```

DLL — Insert at Tail — Python Code

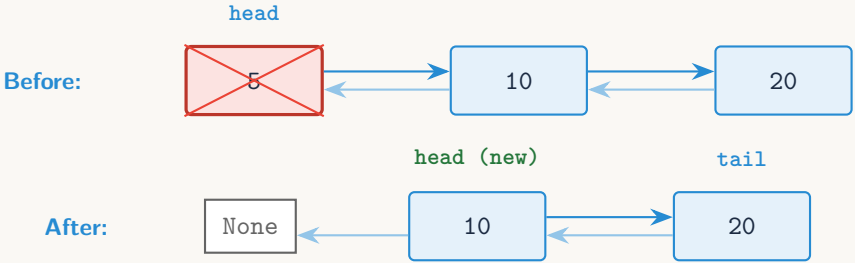
```python
def insert_at_tail(self, data):
    """Insert at the end. O(1) thanks to tail pointer!"""
    new_node = DNode(data)
    if self.tail is None:          # empty list
        self.head = new_node
        self.tail = new_node
    else:
        new_node.prev = self.tail  # link backward
        self.tail.next = new_node  # link forward
        self.tail = new_node       # update tail
    self.size += 1

```

Key Improvement

With a `tail` pointer, insert-at-tail is now $O(1)$!

Compare: $O(n)$ in a singly linked list.

DLL — Delete at Head



DLL — Delete at Head — Python Code

```python
def delete_at_head(self):
    """Remove the first node. O(1)"""
    if self.head is None:
        raise IndexError("List is empty")
    data = self.head.data
    if self.head == self.tail:     # only one node
        self.head = None
        self.tail = None
    else:
        self.head = self.head.next
        self.head.prev = None      # clear backward link
    self.size -= 1
    return data

```

DLL — Delete at Tail — Python Code

```python
def delete_at_tail(self):
    """Remove the last node. O(1)!"""
    if self.tail is None:
        raise IndexError("List is empty")
    data = self.tail.data
    if self.head == self.tail:     # only one node
        self.head = None
        self.tail = None
    else:
        self.tail = self.tail.prev
        self.tail.next = None      # clear forward link
    self.size -= 1
    return data

```

Key Improvement

Delete at tail is $O(1)$ in a DLL!

In a singly linked list it was $O(n)$.

DLL — Insert at Index

```python
def insert_at_index(self, index, data):
    """Insert at given index. O(n)"""
    if index < 0 or index > self.size:
        raise IndexError("Index out of range")
    if index == 0:
        return self.insert_at_head(data)
    if index == self.size:
        return self.insert_at_tail(data)
    new_node = DNode(data)
    current = self.head
    for _ in range(index):
        current = current.next   # walk to position
    # Wire up all four links
    new_node.prev = current.prev
    new_node.next = current
    current.prev.next = new_node
    current.prev = new_node
    self.size += 1

```

DLL — Insert at Index — Visualization

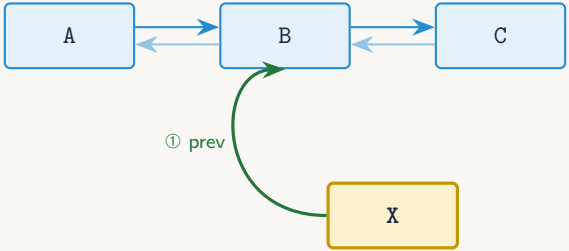
Insert X between B and C:

Four Pointer Updates

(click to begin)

DLL — Insert at Index — Visualization

Insert X between B and C:

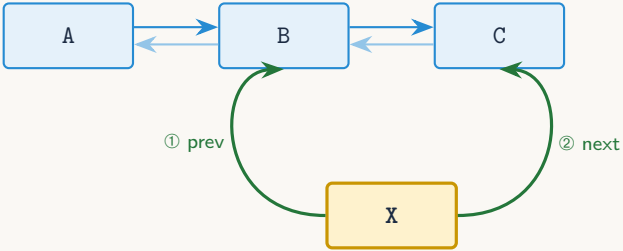


Four Pointer Updates

① `new.prev = B`

DLL — Insert at Index — Visualization

Insert X between B and C:

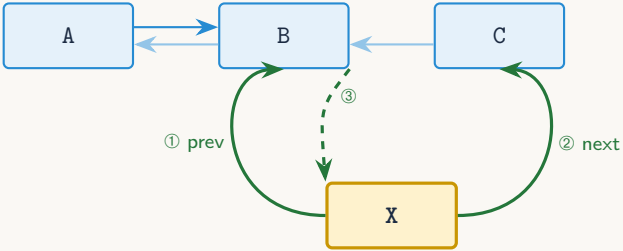


Four Pointer Updates

① `new.prev = B` ② `new.next = C`

DLL — Insert at Index — Visualization

Insert X between B and C:

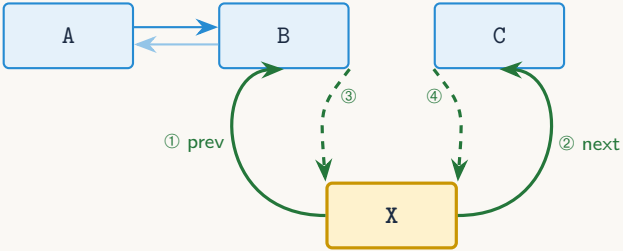


Four Pointer Updates

① `new.prev = B` ② `new.next = C` ③ `B.next = new`

DLL — Insert at Index — Visualization

Insert X between B and C:



Four Pointer Updates

① `new.prev = B` ② `new.next = C` ③ `B.next = new` ④ `C.prev = new`

Part IV: Analysis & Practice

Complexity Comparison — Full Picture

Operation	Python List	Singly LL	Doubly LL
Access by index	$O(1)$	$O(n)$	$O(n)$
Insert at head	$O(n)$	$O(1)$	$O(1)$
Insert at tail	$O(1)^*$	$O(n)$	$O(1)$
Insert at index k	$O(n)$	$O(n)$	$O(n)$
Delete at head	$O(n)$	$O(1)$	$O(1)$
Delete at tail	$O(1)$	$O(n)$	$O(1)$
Delete at index k	$O(n)$	$O(n)$	$O(n)$
Search	$O(n)$	$O(n)$	$O(n)$

* Amortized — occasional resize costs $O(n)$.

When to Use Which?

✓ Use Linked Lists When

- Frequent insertions/deletions at the **beginning**
- You don't need random access
- Data size is unpredictable
- You're building stacks or queues

× Prefer Arrays When

- You need fast **index-based access**
- Data size is known in advance
- Memory efficiency matters
- Cache performance is critical

Real-World Uses of Linked Lists

↺ Undo/Redo Systems

Text editors use a doubly linked list of states.
Move backward (undo) or forward (redo).

♯ Music Playlists

Previous/Next song navigation is naturally a DLL.

♢ Browser History

Back/Forward buttons traverse a linked list of visited pages.

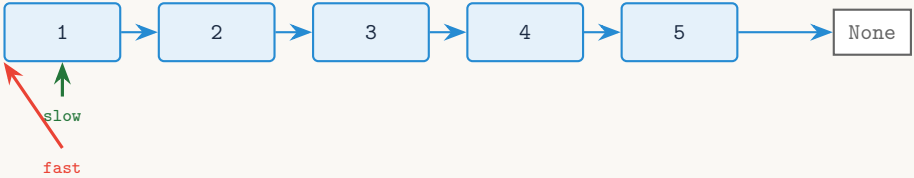
◇ Memory Allocation

Operating systems track free memory blocks using linked lists.

Common Pattern: Two-Pointer (Slow & Fast)

Idea

Use two pointers moving at different speeds to solve problems.



Applications:

- Find the **middle** of a list (slow moves 1, fast moves 2)
- Detect **cycles** in a list (Floyd's algorithm)

Finding the Middle Node

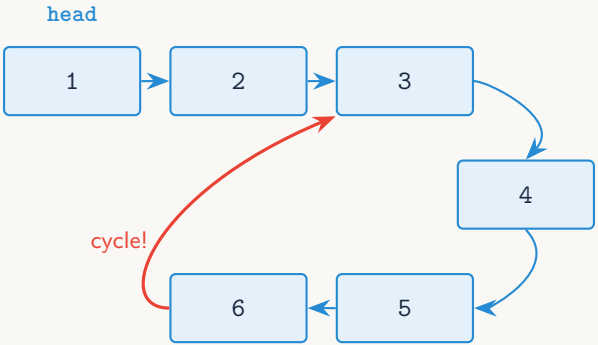
```python
def find_middle(self):
    """Find the middle element using slow/fast pointers."""
    slow = self.head
    fast = self.head
    while fast is not None and fast.next is not None:
        slow = slow.next           # move 1 step
        fast = fast.next.next      # move 2 steps
    return slow.data  # slow is at the middle!

```

Why It Works

When `fast` reaches the end, `slow` has moved exactly half the distance.

Time: $O(n)$, **Space:** $O(1)$ — single pass!

Cycle Detection — Floyd's Algorithm



Algorithm

If `slow` and `fast` ever **meet**, there is a cycle.
If `fast` reaches `None`, there is no cycle.

Common Pitfalls & Debugging Tips

† Pitfall 1: Lost Nodes

Updating pointers in the wrong order can "orphan" nodes.

Fix: Always link the new node first, *then* update the list reference.

† Pitfall 2: Off-by-One

Traversing one node too far (or too few).

Fix: Draw the list on paper and trace your loop.

† Pitfall 3: Forgetting Edge Cases

Empty list, single element, or operating on the last node.

Fix: Test every method with 0, 1, and 2 elements.

† Pitfall 4: DLL Link Mismatch

In a DLL, if `A.next = B` but `B.prev` $\neq$ `A`, your list is broken.

Fix: Always update *both* directions.

Python's Built-In: `collections.deque`

Under the Hood

Python's `collections.deque` is implemented as a **doubly linked list** of fixed-size blocks.

```python
from collections import deque
d = deque([1, 2, 3])
d.appendleft(0)   # O(1) insert at head
d.append(4)        # O(1) insert at tail
d.popleft()        # O(1) delete at head
d.pop()            # O(1) delete at tail
print(d)  # deque([1, 2, 3])

```

Takeaway

In production Python, prefer deque for linked-list-like behavior.
But understanding the internals is what this course is about!

Summary — Key Takeaways

1. A **linked list** stores elements in **nodes** connected by references.
2. A **singly linked list** has one-way links (`next` only).
3. A **doubly linked list** adds `prev` links for two-way traversal.
4. **Insert/delete at head** is $O(1)$ — the main advantage over arrays.
5. **Random access** is $O(n)$ — the main disadvantage.
6. Always handle **edge cases**: empty list, single node.
7. **Pointer order matters** — link new nodes before cutting old links.
8. In Python, `collections.deque` provides DLL-like behavior out of the box.